Contents of the lecture

(1) “while” loop

(2) “for” loop

(3) “repeat” loop

(4) “forever” loop

(d) "while" Loop

```
while (<expression>)
sequential_statement;
```

(i) The value of <expression> is evaluated and if it is true then sequential_statement executes
(ii) The sequential_statement executes till the <expression> is true
(iii) The sequential_statement can be a single statement or a group of statements within "begin…..end"

Example

```
module xyz;
integer mycount;
initial
begin
mycount = 0;
while (mycount <= 5)
#5 mycount = mycount + 1;
end
endmodule
```

(e) "for" Loop

```
for (expr1; expr2; expr3)
sequential_statement;
```

(i) The "for" loop executes as long as the expression expr2 is true

(ii) The sequential_statement can be a single statement or a group of statements within "begin…..end"

(iii) The "for" loop consists of three parts:

(a) An initial condition (expr1)

(b) A check to see the terminating condition is true (expr2)

(c) A procedural assignment to change the value of the control variable (expr3)

(iv) The "for" loop is conveniently used to initialize an array of memory

Example

```
module xyz;
integer mycount;
initial
for (mycount = 0; mycount <=2; mycount = mycount + 1)
#5 mycount = mycount;
endmodule
```

(f) "repeat" Loop

```
repeat (<expression>)
sequential_statement;
```

i. The "repeat" executes the loop a fixed number of times
ii. The expression in "repeat" construct can be a constant, a variable or a signal value (signal means constantly driven signal)
iii. If the expression is a variable or a signal value, it is evaluated only when the loop starts and not during execution of the loop
iv. The sequential_statement can be a single statement or a group of statements within "begin…..end"

Example:

```
repeat (a)
begin
------
end
```

i. There may be chances that variable 'a' might be updated by some statement in "initial" or "always" block
ii. But when "repeat" is executing at that time what ever was the initial value of 'a' will be considered

Example-1

```
module xyz(clk);
  output clk;
  reg clk;
initial
begin
clk = 1'b0;
repeat (2)
#5 clk = ~clk;
end
endmodule
```

(g) "forever" Loop

```
forever
sequential_statement;
```

i. The "forever" construct does not use any expression and executes forever until "$finish" is encountered in the test bench
ii. The "forever" is equivalent to "while" loop for which the expression is always true
iii. The sequential_statement can be a single statement or a group of statements within "begin…..end"

Example

```
while (1 < 2)
begin
……
end
```

Example

```
forever
begin
……
end
```

i. The "forever" loop is typically used along with timing specifier
ii. If delay is not specified then simulator would execute this statement indefinitely without advancing $time
iii. Rest of the design will never be executed

Example:

```
reg clk
initial
begin
clk = 1'b0;
forever
#5 clk = ~clk;
end
```